

Flock-ZK: A Candidate Zero-Knowledge Extension for Batched BLAKE3

Alexander Lee

July 2026

Abstract

We present Flock-ZK, a candidate zero-knowledge extension of Flock for two registered 256-instance BLAKE3 relations. The primary relation proves that 256 hidden 64-byte messages hash to 256 public 32-byte digests:

$$y_i = \text{BLAKE3}(x_i), \quad |x_i| = 64 \text{ bytes}, \quad |y_i| = 32 \text{ bytes},$$

while revealing nothing about those messages beyond what the statement itself implies. The second relation is a benchmark-only batch profile with no public digest binding. BLAKE3 is the first target because it is Flock’s original benchmark, has the most mature circuit in the implementation, and permits a direct comparison with non-ZK Flock [1]. A companion Rust artifact contains the prover, the unchanged family verifier used for both honest and simulated proofs, a public-statement-only simulator, an extractor, Lean lemmas, and replayable certificates.

In the classical programmable random-oracle model, we construct the simulator and derive a complete indistinguishability bound. The earlier polynomial commitment query-rank obligation is closed for every verifier-generated query set: the initial mask columns span all polynomials of degree below 512, so evaluation at any 218 distinct verifier points is surjective by interpolation. For an adversary making at most $Q_H = 2^{64}$ oracle queries, the final bound is $\text{Adv}_{\text{ZK}} < 2^{-118.499}$. This count includes protocol and Merkle queries, transcript hashes, all sixty grinding sites, oracle collisions, and programming freshness. The claim applies only to the registered relations and Flock-ZK verifier. It does not apply to the original Flock verifier, other circuits, or quantum random-oracle attackers.

Knowledge security is separate and incomplete. One term in the available standalone Fiat-Shamir reduction already limits it to at most 55.994 bits at $Q_H = 2^{64}$, before other knowledge errors are added. At the registered batch size, the median twelve-thread Flock-ZK prover time is $6.48\times$ the matched non-ZK path, the median verifier time is $1.28\times$, and the encoded proof is $9.22\times$ larger.

1 Result and scope

Result in one sentence. For the two registry entries in Table 2, and under the classical programmable-random-oracle model, the view produced from a valid witness is computationally indistinguishable from one produced from the public statement alone, with the concrete bound of Theorem 1.

Table 1 states the proof status before the construction is introduced. In particular, the PCS query-rank condition is no longer a fixture-only assumption. Lemma 2 proves it for every query set the registered verifier can generate. The full protocol composition remains a paper proof

supported by executable tests, rather than one end-to-end Lean theorem, and the knowledge theorem remains incomplete.

Table 1: Claim status for the registered Flock-ZK artifact.

Component	Status
Generic affine masking lemmas	Proved in Lean
Registered PIOP determinant	Replayable executable certificate
PCS opening translation	Structural interpolation proof; exhaustive small-domain NTT differential test
Random-oracle ledger	Complete executable count and analytic grinding tail
Full simulator composition	Paper proof and tests; not end-to-end formalized
Knowledge security	Incomplete; current reduction ceiling is below 56 bits

Flock-ZK starts with BLAKE3 for scope, not because the masking method is incompatible with other hashes. A zero-knowledge theorem attaches to an exact relation, circuit layout, transcript, and PCS parameter set. BLAKE3 already had Flock’s most mature circuit and non-ZK benchmark, so it was the smallest target that permits both a complete implementation and a meaningful performance comparison.

The application relation, and the main subject of this paper, is

$$\mathcal{R}_{\text{fd}} = \{((y_1, \dots, y_{256}), (x_1, \dots, x_{256})) : y_i = \text{BLAKE3}(x_i), |x_i| = 64 \text{ bytes}, |y_i| = 32 \text{ bytes}\}.$$

The benchmark-only batch relation uses the same BLAKE3 circuit but has no externally fixed digest list or hash output. Its statement exposes only the batch shape, so the prover is free to choose the trace. It is retained solely to measure the cost of adding masking to Flock’s original path. Application claims in the rest of the paper concern $\mathcal{R}_{\text{fd}}$. In both profiles the R1CS shape, packing convention, PCS parameters, and circuit digest are fixed by the registry.

Throughout the paper, *computational zero knowledge* has its standard classical-ROM meaning: for every classical probabilistic polynomial-time distinguisher making at most Q_H oracle queries, the joint view of an honest proof and its oracle answers is computationally indistinguishable from the view produced by a simulator that knows only the statement. The simulator may program points that the distinguisher has not already queried. This is the IOP and Fiat–Shamir setting of Ben-Sasson, Chiesa, and Spooner [2]; the algebraic masking step is in the spirit of zero-knowledge sumcheck [3].

Table 2: The complete implementation scope of the ZK claim.

Profile	Public input	Circuit / PCS	Role
Fixed-digest preimage	256 digests	$m = 22$, rate 1/2, batch log 6	Application profile
Benchmark-only batch	batch size 256	$m = 22$, rate 1/2, batch log 6	Performance control

The scope is deliberately narrow. The preimage circuit hashes one message of exactly 64 bytes and pins the BLAKE3 IV, counter, block length, and `CHUNK_START`, `CHUNK_END`, and `ROOT` flags. Unknown circuit digests, batch sizes, and PCS parameters are rejected before proving. SHA-256 statements, Keccak statements, longer BLAKE3 messages, hash chains, Merkle-path statements, recursion, side channels, and quantum random-oracle queries are not covered. This is an absence

of a family-specific proof, not a claim that the construction cannot be adapted. Section 10 lists the obligations that must be re-established for each new family.

Construction at a glance. The argument follows the data seen by the verifier:

1. the BLAKE3 computation is written as an R1CS assignment z satisfying $(Az) \circ (Bz) = Cz$;
2. zerocheck reduces all multiplication constraints to evaluations at a transcript-derived point;
3. lincheck ties those evaluations back to the public matrices A, B, C and the same committed assignment z ;
4. ring switching converts the bit-level evaluation claims into packed extension-field claims, which Ligerito commits to and opens;
5. independent masks randomize every witness-dependent value while preserving the values that the verifier checks; and
6. the simulator samples those randomized values from the public statement and programs only fresh, domain-separated oracle points.

Sections 2–6 develop this construction in order. Section 7 measures its cost. Sections 8 and 9 treat extraction separately because witness privacy and knowledge soundness are different properties. Section 11 states exactly which obligations are proved in Lean, checked by executable certificates, or left as explicit assumptions.

Assumption 1 (Classical programmable random oracle). All injectively framed SHA-256 calls are modelled as one random function $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$. The adversary makes classical queries, and the simulator may program a point only before that point is queried. No QROM claim is made.

2 Relation and construction

Let z be the packed binary assignment for the block-diagonal BLAKE3 R1CS, embedded in $\mathbb{F}_{2^{128}}$, and write

$$a = Az, \quad b = Bz, \quad c_z = Cz.$$

The R1CS is satisfied when $a \circ b = c_z$. Zerocheck treats the vectors as multilinear polynomials and reduces the assertion $\widehat{a}(y)\widehat{b}(y) + \widehat{c}_z(y) = 0$ on the Boolean cube to terminal evaluations at a random point. Each round folds one or more variables and leaves a smaller claim. A false residual polynomial reaches the terminal identity only on a small exceptional set of challenges.

Zerocheck alone does not show that the prover used $a = Az$, $b = Bz$, and $c_z = Cz$. Lincheck supplies that link. It randomly combines the public matrix relations, asks for a partial evaluation of z , and reduces consistency to evaluation claims on the same committed assignment. Thus zerocheck checks the multiplicative constraints, while lincheck prevents the prover from inventing unrelated vectors that merely satisfy their product equation.

The resulting claims are opened with the ring-switched Ligerito PCS [4, 5]. The companion cheat sheet gives a high-level account of Ligerito, its relationship to Ligerito, and ring switching.

For the fixed-digest family, the verifier derives an additional packed opening claim from the public digest list. Before the first challenge, the transcript binds that list, the padding rule, the witness layout, the circuit matrices, the parameter set, and a fresh 32-byte proof nonce.

The ZK prover adds four independent mask sources: the zerocheck mask P , the lincheck mask S , and the round-one masks S_c, S_h . The witness and mask commitments use independent hiding randomness, denoted μ and g . Every stream, including the proof nonce, is derived from a per-proof DRBG under a distinct label.

The main zerocheck mask is

$$\gamma P(X)Q^*(X), \quad Q^*(X) = q_0 + \sum_j q_j X_j.$$

Here P is a fresh witness-free polynomial, committed before the verifier samples γ . Its coefficients are uniform in $\mathbb{F}_{2^{128}}$ on a translated subcube with $d_{\log} = 12$ variables. The polynomial Q^* is fixed, public, affine, and nonzero on the Boolean cube. Multiplication by Q^* gives a degree-two mask with enough independent directions to randomize both coordinates of each degree-two zerocheck round message; Proposition 1 makes that statement quantitative. The challenge γ prevents the prover from choosing P after seeing how it would need to cancel an invalid computation. At the terminal point ρ , the verifier checks

$$\widehat{a}(\rho)\widehat{b}(\rho) + \gamma P(\rho)Q^*(\rho),$$

and authenticates $P(\rho)$ against its earlier commitment. The mask therefore changes the visible proof distribution without becoming a free correction to the relation. The separate masks S_c, S_h cover the first zerocheck message, and S covers lincheck.

3 One random-oracle boundary

In Flock-ZK, Fiat–Shamir challenges, Merkle commitments, and proof of work are all implemented through the one random oracle of Assumption 1, but domain separation prevents them from using the same input. Each input begins with a role byte. A Merkle input also binds the wire-protocol identifier, proof nonce, commitment channel, recursion depth, node level, node index, payload length, and payload. A proof-of-work input is

$$\text{ROLE_POW} \parallel \text{state} \parallel \text{nonce}.$$

The proof nonce is absorbed immediately after the statement and before the first challenge.

This framing is part of the proof, not merely an encoding convention. It lets the extractor identify witness-leaf queries without relying on their order, prevents simulator programming from aliasing transcript or Merkle inputs, and makes proofs from different nonces or commitment channels use different oracle points. Native, recording, and programming backends implement the same frames. When no point is programmed they return identical bytes. Scalar and four-way SIMD implementations are differentially tested against the one-shot definition for every supported tail shape.

4 The PIOP translation

Fix the post-commitment challenge tuple χ . Let D be the coefficient space of P . If $n = m - 6$ is the number of masked zerocheck rounds, define

$$R_\chi : D \longrightarrow \mathbb{F}_{2^{128}}^{2n}$$

to be the map from mask coefficients to the $2n$ round-message coordinates. Let

$$L_\chi : D \longrightarrow \mathbb{F}_{2^{128}}^2$$

map the same coefficients to the two mask values revealed later: the initial masked claim and $P(\rho)$. The kernel $\ker L_\chi$ is simply the set of masks that change neither revealed value. Once $L_\chi(P)$ is fixed, these are the only mask changes still available. Their effect on the round messages is the subspace $R_\chi(\ker L_\chi)$.

Lemma 1 (Conditioned image dimension). *Let $N = \dim D$. If*

$$\text{rank} \begin{bmatrix} R_\chi \\ L_\chi \end{bmatrix} = 2n, \quad \text{rank } L_\chi = 2,$$

then

$$\dim R_\chi(\ker L_\chi) = 2n - 2.$$

Proof. The stacked-rank assumption gives $\dim(\ker R_\chi \cap \ker L_\chi) = N - 2n$, while $\text{rank } L_\chi = 2$ gives $\dim \ker L_\chi = N - 2$. Rank-nullity for R_χ restricted to $\ker L_\chi$ now gives

$$\dim R_\chi(\ker L_\chi) = (N - 2) - (N - 2n) = 2n - 2.$$

□

For the registered parameters, $m = 22$ and $n = 16$, so R_χ has 32 output coordinates. The sumcheck recurrence makes the two rows of L_χ dependent on the rows of R_χ , so the stacked map has rank at most 32. To prove that it has rank 32, it is enough to find a square 32×32 submatrix, called a *minor*, with nonzero determinant.

The symbolic evaluator builds the entries of this matrix as polynomials in the 32 verifier challenges. It follows the same straight-line recurrences as the concrete evaluator, while tracking either field values or conservative multivariate degrees. It contains no challenge-dependent inversion. The selected minor evaluates to a nonzero field element at one recorded challenge tuple, proving that its determinant is not the zero polynomial. Summing the per-row degree bounds gives total degree at most 720. Thus 720 is not a constraint count or a fitted constant; it is a conservative degree bound for this determinant.

Proposition 1 (Registered field-mask coverage). *For uniformly sampled $\chi \in \mathbb{F}_{2^{128}}^{32}$, the registered Flock-ZK shape satisfies the hypotheses of Lemma 1, except with probability*

$$\Pr_\chi[\text{coverage failure}] \leq \frac{720}{2^{128}} < 2^{-118.508}.$$

Consequently, after the two revealed mask values are fixed, the remaining mask freedom covers all $2n - 2 = 30$ independent round-message directions.

Proof. The nonzero minor has total degree at most 720. Schwartz–Zippel bounds the probability that it vanishes at a uniform challenge tuple by $720/|\mathbb{F}_{2^{128}}| = 720/2^{128}$ [6, 7]. Outside that zero set, the minor gives rank 32 for the stacked map, the separately checked leakage map has rank 2, and Lemma 1 applies. Section 11 records how the matrix, nonzero evaluation, and degree vector are replayed. □

Proposition 1 proves the PIOP masking step. It says nothing by itself about privacy at the PCS opening boundary; that is the next step.

5 The PCS translation

Write the initial combined opening vector as

$$F = (\mu \parallel z) + c g,$$

where $c \in \mathbb{F}_{2^{128}}$ is sampled after commitment. Let d be a witness difference that preserves every public opening claim. The aim is to replace z by $z + d$ without changing anything the verifier opens.

Lemma 2 (Initial query rank). *Let K be the number of low-mask symbols in one initial encoding lane. For any set S of $q \leq K$ distinct positions in the additive-NTT domain, the map from those K symbols to the evaluations at S has row rank q . The registered verifier has $K = 512$, samples without replacement, and opens $q = 218$ positions. Hence every query set it can generate has full row rank.*

Proof. The first K normalized LCH novel-basis elements have degrees $0, 1, \dots, K - 1$. They therefore form a basis for the polynomials of degree less than K . Given arbitrary target values at the q distinct domain points, interpolation produces a polynomial of degree less than $q \leq K$ with exactly those values. Its coordinates in the novel basis give a preimage of the target vector. Thus the evaluation map is surjective and its matrix has row rank q . $\square$

Concretely, the registered initial domain has 1,024 positions. The verifier's sampler returns exactly 218 indices from this domain without replacement, and the low mask contributes 512 coefficients to each of the 64 encoding lanes. Thus its output always satisfies $S \subseteq \{0, \dots, 1023\}$ and $|S| = 218 < 512$. The rank statement is about all such sets, not a sampled transcript.

Lemma 3 (Opening-preserving translation). *Suppose $c \neq 0$ and the registered verifier produces the query set. There is an affine bijection on the uniform PCS masks such that replacing z by $z + d$ leaves the queried initial rows and the entire combined vector F unchanged. Consequently every recursive codeword, root, authentication path, sumcheck value, and out-of-domain sample is unchanged.*

Proof. Set

$$\delta g_{\text{top}} = c^{-1} d.$$

By Lemma 2, choose $\delta \mu$ to solve the queried initial-level encoding equations, and set

$$\delta g_{\text{lo}} = c^{-1} \delta \mu.$$

Because the field has characteristic two, $d + c \delta g_{\text{top}} = 0$ and $\delta \mu + c \delta g_{\text{lo}} = 0$. Thus $\delta F = 0$ globally. The defining equations for $\delta \mu$ also hold the separately opened initial rows fixed. Translation by these deltas is a bijection on uniform masks, so it preserves their distribution. $\square$

The registered initial leaf contains 64 pairs of field elements, one element from the masked message and one from its blinder. Since $218 < 512$, conditioning on all opened positions leaves a fresh leaf with 128 independent elements of $\mathbb{F}_{2^{128}}$. The minimum conditional entropy used at the Merkle boundary is therefore $128 \cdot 128 = 16,384$ bits. No recursive entropy charge is needed because Lemma 3 makes the recursive codewords identical. The exceptional event $c = 0$ has probability 2^{-128} . The implementation rejects duplicate, out-of-domain, and oversized query sets, and an exhaustive small-domain test compares the structural certificate with the actual additive-NTT matrices for every admissible query subset.

6 Simulator and concrete ZK bound

The simulator is an explicit algorithm, not an existence claim. Its input is the registered setup and public digest list. Its type contains no messages or other witness-bearing object. It has private coins and access to the programmable oracle of Assumption 1.

Simulator $\text{Sim}(x; \omega, \mathcal{H})$. For public statement x and private coins ω :

1. Sample 256 unrelated 64-byte messages and compute their ordinary BLAKE3 trace. Replace only the public-output words by the digests in x , then recompute the A -, B -, and lincheck vectors from this patched assignment. This assignment satisfies the public digest opening but is not an R1CS witness.
2. From independently labelled streams, sample the 32-byte proof nonce, the masks P, S, S_c, S_h , the commitment masks μ, g , and the challenges $z, \gamma, \rho_1, \dots, \rho_{16}$. Restart if $\rho_{16} \in \{0, 1\}$.
3. Absorb the registered statement and proof nonce and emit commitments in verifier order. All commitment and Merkle calls use the framing of Section 3.
4. Emit the zerocheck in one pass. Sample every free round coordinate, solve one coordinate of the first C -message so that its interpolation equals the committed C -evaluation, and solve the last $G(\infty)$ coordinate so that the terminal identity holds. Immediately before each corresponding Fiat–Shamir sample, program the current framed oracle input to return z , then γ , then $\rho_1, \dots, \rho_{16}$. Restart if any input was previously queried.
5. Run lincheck, ring switching, and the Ligerito opening honestly on the patched committed vector. Perform proof of work in transcript order at all sixty registered sites. Assemble and return the ordinary Flock-ZK commitment and proof.

The unmodified Flock-ZK verifier accepts the output. The implementation also checks the negative control: sending the same patched assignment through the honest prover is rejected.

For $m = 22$, zerocheck has $m - 6 = 16$ multilinear fold rounds. The points for z, γ , and the sixteen values ρ_j are the only programmed inputs, which explains the constant 18. Lincheck and the PCS challenges remain honest oracle samples.

Let $\text{View}_0(x, w)$ be the joint distribution of an honest proof and all oracle answers for valid witness w , and let View_i denote the distribution after hybrid G_i . The following lemmas identify each change.

Lemma 4 (Hybrid G_0 to G_1 : PIOP translation). *Outside the zero set of the registered determinant, View_0 and View_1 are identical after translating the PIOP mask coins from the honest assignment to the patched assignment. The translation is a bijection on each affine fibre determined by the two opened mask values. The exceptional probability is at most $720/2^{128}$.*

Proof. Proposition 1 gives every unexposed round-message direction after conditioning on the two exposed values. Translation in this full image is a bijection on uniform mask coins, so the resulting messages have exactly the same distribution. Schwartz–Zippel charges the determinant’s zero set. $\square$

Lemma 5 (Hybrid G_1 to G_2 : PCS translation). *Conditioned on $c \neq 0$, View_1 and View_2 have identical algebraic openings after the affine translation of μ and g . The charge is $1/2^{128}$.*

Proof. Lemma 2 makes the initial system solvable for every verifier query set, and Lemma 3 gives the bijection. It fixes the queried initial rows and F globally, hence every recursive algebraic value. Only $c = 0$ prevents the translation. $\square$

Lemma 6 (Hybrid G_2 to G_3 : Merkle boundary). *Replace the hidden initial codeword leaves by their translated values and resample the corresponding framed oracle outputs lazily. The two views are identical unless an oracle query guesses a hidden sibling input or two oracle outputs collide. For total oracle budget Q_T , the charge is at most $Q_T/2^{16384} + \binom{Q_T}{2}/2^{256}$.*

Proof. Every opened initial row is fixed and every recursive codeword is identical. An unopened initial leaf has at least 16,384 bits of conditional entropy, so a union bound charges $Q_T/2^{16384}$. Lazy sampling of a 256-bit function adds the stated collision term. $\square$

Lemma 7 (Hybrid G_3 to G_4 : programmed challenges). *Replace the eighteen honest Fiat–Shamir samples by eighteen uniform values chosen in advance and program their exact framed inputs in transcript order. The distributions are identical unless the adversary queried one of those inputs first. The charge is at most $18Q_H/2^{256}$.*

Proof. A fresh random-oracle answer is uniform whether sampled on demand or chosen first and installed later. The proof nonce makes each programmed transcript prefix a fresh 256-bit target. A union bound over eighteen points and Q_H adversarial queries gives the claim. $\square$

Lemma 8 (Hybrid G_4 to G_5 : public-statement simulator). *Outside $\rho_{16} \in \{0,1\}$, the one-pass solve above produces exactly View_4 using only the public statement. The charge is $2/2^{128}$, and View_5 is the output of Sim .*

Proof. Each solved message is one affine constraint with a nonzero pivot. Choosing the remaining coordinates uniformly and solving the pivot is a bijection with the corresponding conditioned honest distribution. The final denominator is $\rho_{16}(1 + \rho_{16})$, which vanishes only at zero and one. The first-round Lagrange weights cannot all vanish because they sum to one. $\square$

Table 3: Hybrid steps from an honest to a simulated transcript.

Hybrid	Replacement	Charge
G_0	honest proof	0
G_1	translate the PIOP mask	$720/2^{128}$
G_2	translate μ and g	$1/2^{128}$
G_3	cross the Merkle boundary	$Q_T/2^{16384} + \binom{Q_T}{2}/2^{256}$
G_4	program fresh challenges	$18Q_H/2^{256}$
G_5	solve the terminal messages	$2/2^{128}$

The oracle ledger counts calls made by one simulator proof and one verification of that proof. The deterministic counts come from a recording backend. The geometric number of prover grinding attempts is replaced by an analytic cap, so the security statement does not depend on one lucky benchmark run.

There are twelve nonzero grinding sites per PCS opening and five openings. The per-opening difficulties are

$$(10, 9, 8, 7, 6, 5, 4, 4, 3, 2, 1, 1).$$

Table 4: Random-oracle ledger for one registered proof.

Call class	Bound
Simulator Merkle leaves / internal nodes	13,440/13,425
Simulator transcript and state calls	2,554
Verifier Merkle leaves / internal nodes	1,975/5,915
Verifier transcript and state calls	2,579
Verifier grinding checks	60
Prover grinding candidates, outside the tail event	957,910
Total protocol queries Q_P	997,858

For a site of difficulty b_s , cap its trials at

$$A_s = \left\lceil 2^{b_s} (128 \ln 2 + \ln 60) \right\rceil.$$

Since $(1 - 2^{-b_s})^{A_s} \leq e^{-A_s/2^{b_s}}$, a union bound over the sixty sites gives failure probability at most 2^{-128} . Summing the caps gives 957,910 candidate hashes. Together with the non-grinding calls in Table 4, this gives $Q_P = 997,858$. Set

$$Q_T = Q_H + Q_P.$$

The honest and simulated provers traverse the same transcript, commitment, and opening schema, so Q_P bounds either experiment and its verification. Programming writes are not counted as queries; each programmed point is later read by the transcript and is already included in the 2,554 state calls. The term $\binom{Q_T}{2}/2^{256}$ ranges over every pair of adversarial and protocol queries, including cross-collisions between them. Domain separation prevents role aliasing at the input level. The collision term covers equal 256-bit outputs, while the programming term separately covers an adversary that queries a programmed input before the simulator does.

The three algebraic numerators are 723, 18, and 2: the determinant bad set, $c = 0$, and the two rejected terminal challenges. Adding the grinding tail, programming, sibling, and collision terms gives the complete bound

$$\varepsilon_{\text{ZK}}(Q_H) = \frac{723}{2^{128}} + \frac{18Q_H}{2^{256}} + \frac{Q_T}{2^{16384}} + \frac{\binom{Q_T}{2}}{2^{256}} + \frac{1}{2^{128}}.$$

At $Q_H = 2^{64}$,

$$-\log_2 \varepsilon_{\text{ZK}}(2^{64}) = 118.499158 \dots$$

If the Merkle, collision, and grinding-tail terms are omitted, the algebraic and programming component alone is 118.502 bits. It is not the final ZK figure; 118.499 bits is the result after the complete classical-ROM ledger is included.

Theorem 1 (Registered classical-ROM ZK bound). *Under Assumption 1, for either registry entry in Table 2 and every classical distinguisher making at most Q_H oracle queries,*

$$\text{Adv}_{\text{ZK}} \leq \varepsilon_{\text{ZK}}(Q_H).$$

The simulator uses only the public statement, and the verifier is the same Flock-ZK verifier used for honest proofs. The original non-ZK Flock verifier is a different protocol and is not covered. In particular, at $Q_H = 2^{64}$, $\text{Adv}_{\text{ZK}} < 2^{-118.499}$.

This theorem is in the classical programmable-ROM. It is not a QROM theorem, and it does not turn the ideal-oracle argument into a standard-model theorem for SHA-256.

7 Performance

Table 5 compares the registered Flock-ZK path with the non-ZK Flock path at their common supported batch size. Both paths use the same 256 BLAKE3 compression inputs and benchmark commit `a3da544ea88042ab14e69cdf0dc5663efa0cc1c3`. Measurements used an Apple M4 Pro with 12 logical CPUs and 24 GiB of memory, Rust 1.97.1, LLVM 22.1.6, the optimized Cargo benchmark profile, and no `RUSTFLAGS`. Each number is the median of twenty trials after one verified warm-up; the value in parentheses is the median absolute deviation.

Table 5: Twelve-thread cost for 256 BLAKE3 compressions. Times are median (median absolute deviation).

Metric	Non-ZK Flock	Flock-ZK	ZK / non-ZK
Prover time per batch	4.511 (0.180) ms	29.225 (0.215) ms	6.48×
Prover throughput	56,745/s	8,760/s	0.154×
Verifier time per batch	9.368 (0.082) ms	11.979 (0.104) ms	1.28×
Encoded proof size	265.67 KiB	2,448.98 KiB	9.22×
Peak incremental heap	2.70 MiB	9.02 MiB	3.34×

The prover ranges were 4.255–4.870 ms for non-ZK Flock and 28.254–29.770 ms for Flock-ZK. The verifier ranges were 9.086–9.520 ms and 11.825–12.559 ms. On one thread, median proving took 12.121 ms and 43.998 ms respectively, a factor of $3.63\times$. The benchmark reports incremental live heap allocated by the measured prover, not whole-process resident memory.

Both verifier measurements run in the same dedicated serial worker pool. This constraint matters: allowing only the Flock-ZK wrapper to use the global Rayon pool can make it appear faster than non-ZK verification even though the paths are not being measured under the same concurrency policy. Under the matched policy, Flock-ZK verification is $1.28\times$ slower. Its additional work comes from the masked transcript, framed commitments, and blinded PCS opening.

The original Flock paper [1] reported less than $250\times$ native overhead at the batch size that maximized throughput. It benchmarked powers of two through $2^{18} = 262,144$ BLAKE3 compressions and reached its peak single-thread throughput at 2^{16} ; the construction did not state a hard maximum batch size. Those figures use much larger batches, different hardware, and an earlier implementation, so they are context rather than a direct baseline for Table 5. In particular, multiplying 250 by 6.48 would not be a measured Flock-ZK overhead.

The table measures the implementation candidate as it exists; it does not claim that the observed factor is an intrinsic cost of zero knowledge. The pinned benchmark artifact contains all twenty samples’ summary statistics, compiler identity, flags, hardware, and the exact commit for both paths.

8 Extraction and simulation extractability

Zero knowledge means that a proof reveals nothing about the witness beyond the truth of the statement. Knowledge soundness asks the complementary question: can a successful prover be used to recover a witness? The two properties use different arguments and have different concrete bounds.

The recording oracle observes the framed leaf queries used to construct the witness commitment. Given a complete valid codeword, the extractor rebuilds the interleaved word, applies the

inverse NTT lane by lane, re-encodes the candidate, checks the configured unique-radius condition, and returns the witness half. The system-specific step then reads the 512 message bits of each instance and independently verifies

$$\text{BLAKE3}(x_i) = y_i.$$

Honest recorded commitments yield the true preimages; simulated commitments do not extract.

The executable decoder is an exact-word decoder and candidate validator. It is not a complete Reed–Solomon decoder for every noisy word inside the theoretical radius. The general noisy-word step therefore comes from the Ligerito proximity and unique-decoding argument, not from the current decoder routine.

For simulation extractability, the simulator records

$$(statement\ digest, proof\ nonce, protocol\ identifier)$$

for each simulated proof. Extraction refuses every recorded prefix and is attempted only on a fresh one. The implemented property is therefore *fresh-prefix weak simulation extractability*. Same-prefix simulation extractability is not claimed.

9 Knowledge-security limit

The knowledge ledger contains an interactive L0 fold term

$$\kappa_\Gamma = \frac{257}{2^{128}},$$

or 119.994 bits. The classical Fiat–Shamir reduction multiplies this term by $Q_H + 1$. At $Q_H = 2^{64}$ this term alone becomes

$$-\log_2((Q_H + 1)\kappa_\Gamma) = 55.994 \dots \text{ bits.}$$

Thus 55.994 bits is a ceiling imposed by one term, not a completed lower guarantee for the whole argument. Every additional positive knowledge-error term can only reduce the final number of provable bits.

The 64-bit loss already occurs in the classical reduction. It comes from guessing which of the adversary’s Q_H oracle queries carries the challenge used in the successful proof, which contributes the factor $Q_H + 1$. No quantum adversary is needed for this loss. Thus “classical loss” means that the stated standalone knowledge theorem is already only 55.994 bits against classical attackers. It does not mean that the protocol is post-quantum secure. The paper gives no quantum-random-oracle reduction and therefore makes no knowledge-security claim against quantum adversaries.

The standalone implementation profile must be labelled *100-bit conjectured classical knowledge security*. Its reduction ceiling is 55.994 bits before the remaining terms are added. A protocol with an unbiased post-commitment beacon could avoid this particular Fiat–Shamir query loss, and a wider challenge field could increase the margin. Neither change is implemented or claimed here. Grinding, a small- Q_H convention, generic parallel repetition, and deleting one fold challenge do not repair the standalone bound.

10 Extending the ZK construction

The masking code is reusable, but the ZK claim does not transfer automatically to another hash, circuit, batch size, or PCS shape. A new family such as SHA-256 or Keccak requires all of the following steps:

1. define the exact relation, public inputs, padding, and fixed hash parameters;
2. bind the circuit digest, witness layout, PCS parameters, and complete transcript schema;
3. regenerate the symbolic PIOP map and prove a nonzero determinant with an explicit all-challenge degree bound;
4. prove that the PCS mask translation is solvable for every query set the Flock-ZK verifier can derive, and recompute the unopened-leaf entropy;
5. construct a simulator whose interface accepts only the public statement and whose output is accepted by the unchanged family verifier;
6. enumerate every verifier-visible field and prove the hybrid composition, including random-oracle programming and grinding order;
7. implement the relation-specific extractor and recompute the complete ZK and knowledge-error ledgers; and
8. add a fail-closed certificate entry only after the family-specific negative controls and end-to-end certification pass.

The existing SHA-256 and Keccak encoders do not satisfy these obligations merely by existing. The algebraic masking strategy is expected to be reusable, but those statement families remain outside the ZK claim until their own certificates, simulators, extractors, and error ledgers are established.

11 Implementation evidence and limitations

The companion implementation and evidence suite is available here:

<https://github.com/partylikeits1983/flock/tree/zk-flock>.

The registered PCS parameters are $m = 22$, rate $1/2$, batch log 6, and 218 initial queries. The benchmark-only batch circuit digest is

4a398ca2e73d9d1f70611bd6a2a408b0404aabddcf6cf75c29cfa412f72f8423,

and the fixed-digest circuit digest is

e71bde2b053b1b697412d2d30c2e2e57f7b5f1822484fcbcc15c89ea202890d5.

The registry rejects any other digest or parameter tuple before proving.

The single reproduction command is

`scripts/zk-paper-reproduce.sh`.

It runs the certificate suite and benchmark harness. Performance numbers remain hardware-dependent; the harness prints the commit, machine, compiler, flags, medians, dispersion, and ranges needed to interpret a rerun. The certificate artifacts are kept under `docs/artifacts` in the repository.

The evidence is deliberately layered. Lean proves reusable mathematical statements. Rust constructs and replays the matrices and translations for the registered parameters. The paper

Table 6: Formalization and executable-evidence status.

Claim	Machine support	Status
Uniform affine masking and bad-set composition	Generic Lean theorems	Proved in Lean; no protocol layout is encoded there
Schwartz–Zippel degree budget	Generic Lean theorem; Rust supplies the determinant data	Generic implication proved in Lean
Registered PIOP coverage	Symbolic Rust matrix, nonzero minor, and replayed degree vector	Protocol-specific certificate, not an end-to-end Lean theorem
PCS opening translation	Structural interpolation lemma, exact Rust translator, and NTT differential tests	Proved for every distinct registered query set with $q \leq 512$
Random-oracle ledger	Recording backend, framed-call classifier, and analytic grinding cap	Complete for the registered profile at the stated query budget
Full ZK composition	Simulator tests, hybrid ledger, and paper argument	Classical-ROM theorem under the stated assumptions; not one Lean theorem
Extraction	Recording oracle and executable candidate decoder	Exact-word path implemented; general noisy decoding remains theoretical

supplies the protocol-specific composition and states the random-oracle assumptions. Table 6 separates these roles.

Thus Theorem 1 is not presented as a single end-to-end Lean proof. Lean checks its generic masking, bad-set, and degree-budget ingredients. The exact Flock-ZK instantiation also relies on replayable Rust certificates, the paper proofs above, and the classical programmable-random-oracle model.

The executable evidence ties the numerical claims to the implementation, but does not replace cryptographic review. The remaining limits are:

- The ZK theorem is classical-ROM only; there is no QROM analysis.
- The standalone knowledge reduction over $\mathbb{F}_{2^{128}}$ is capped at 55.994 bits at $Q_H = 2^{64}$ before other terms are added. This is not a completed 100-bit theorem.
- The executable decoder validates candidates but does not implement general arbitrary-error Reed–Solomon decoding.
- Only the two exact registry entries are enabled. Other Flock statement families remain non-ZK.
- Side-channel resistance, recursive composition, and same-prefix simulation extractability are out of scope.

Within these limits, the conclusion is precise. Flock-ZK is a credible, implemented, narrowly scoped ZK candidate for the registered BLAKE3 relations. It has a public-statement-only simulator whose proofs are accepted by the ordinary Flock-ZK verifier, a structural proof of PCS query solvability, and a complete classical-ROM error ledger. It is not a blanket result that Flock, arbitrary Flock circuits, or the original Flock verifier are zero knowledge.

References

- [1] Succinct Labs. *Flock*. Historical paper at repository commit 73f7202. <https://github.com/succinctlabs/flock/blob/73f7202/paper/flock-paper.pdf>.
- [2] E. Ben-Sasson, A. Chiesa, and N. Spooner. *Interactive Oracle Proofs*. TCC 2016; IACR ePrint 2016/116.
- [3] A. Chiesa, M. A. Forbes, and N. Spooner. *A Zero Knowledge Sumcheck and its Applications*. IACR ePrint 2017/305.
- [4] A. Novakovic and G. Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. IACR ePrint 2025/1187.
- [5] B. E. Diamond and J. Posen. *Polylogarithmic Proofs for Multilinears over Binary Towers*. IACR ePrint 2024/504.
- [6] J. T. Schwartz. *Fast Probabilistic Algorithms for Verification of Polynomial Identities*. Journal of the ACM 27(4), 701–717, 1980.
- [7] R. Zippel. *Probabilistic Algorithms for Sparse Polynomials*. EUROSAM 1979, LNCS 72, 216–226.